



OPEN ACCESS

EDITED BY
Chengbo Wang,
Xidian University, China

REVIEWED BY
Zeguo Zhang,
Guangdong Ocean University, China
Jiong Li,
Shanghai Maritime University, China

*CORRESPONDENCE
Mingyu Wang
✉ vicsee@163.com

RECEIVED 31 December 2025
REVISED 25 January 2026
ACCEPTED 30 January 2026
PUBLISHED 27 February 2026

CITATION
Wang Y, Wang M, Shi J, Ni Z
and Li G (2026) eAodeMar: a lightweight
and real-time occluded marine vessel
detection network for embedded marine
platforms.
Front. Mar. Sci. 13:1778827.
doi: 10.3389/fmars.2026.1778827

COPYRIGHT
© 2026 Wang, Wang, Shi, Ni and Li. This
is an open-access article distributed under
the terms of the [Creative Commons
Attribution License \(CC BY\)](#). The use,
distribution or reproduction in other
forums is permitted, provided the
original author(s) and the copyright
owner(s) are credited and that the
original publication in this journal is
cited, in accordance with accepted
academic practice. No use, distribution
or reproduction is permitted which does
not comply with these terms.

eAodeMar: a lightweight and real-time occluded marine vessel detection network for embedded marine platforms

Yuanyuan Wang^{1,2}, Mingyu Wang^{1,2*}, Jianqiang Shi^{1,2,3},
Zuo Ni^{1,2} and Guobin Li¹

¹International Navigation College, Hainan Tropical Ocean University, Sanya, China, ²Yazhou Bay Innovation Institute of Hainan Tropical Ocean University, Sanya, China, ³Weihai Guangtai Airport Equipment Co., Ltd., Weihai, China

Introduction: The detection of occluded marine vessels is critical for the safe navigation and operation of unmanned surface vehicles (USVs). While image-based detection methods have achieved substantial accuracy, their high computational and memory requirements prohibit deployment on resource-constrained embedded platforms. To address this, we propose eAodeMar (efficient AodeMar), a lightweight version built upon our prior AodeMar model, specifically designed for efficient occluded marine vessel detection.

Methods: The efficiency of eAodeMar is achieved by integrating Ghost convolution modules into both the backbone and the feature fusion network, significantly reducing model parameters and computational load while maintaining accuracy. To ensure practical applicability, the optimized model is deployed on an embedded GPU platform (Jetson Xavier NX), incorporating dedicated structural refinement and inference acceleration techniques.

Results: Extensive experiments on the public MVDD13 dataset demonstrate that eAodeMar reduces parameter count and computational load by 7.00% and 0.89%, respectively, with only a marginal accuracy drop of 0.42%, while achieving a remarkable 42.12% improvement in inference speed. When deployed on the Jetson Xavier NX device, it attains a real-time detection rate of 28.57 FPS on the SMD video stream.

Discussion: These comprehensive results validate that eAodeMar effectively balances high precision with high efficiency in occlusion-prone maritime environments. The model demonstrates strong potential for real-world ocean engineering applications, offering a practical solution for real-time detection on embedded systems.

KEYWORDS

embedded platform, ghost convolution, light-weight network, marine vessel detection, real-time detection, unmanned surface vehicle

1 Introduction

The advancement of unmanned platforms, including unmanned surface vessels (USVs) and aerial drones, has significantly transformed modern ocean observation and marine engineering operations (Liu et al., 2025). These platforms increasingly rely on embedded visual perception systems to achieve automated, real-time monitoring of maritime targets,

particularly ships. This capability is critical for maritime traffic management (Gao et al., 2025), navigational safety, early hazard warning, environmental surveillance (Song et al., 2024), and collision avoidance (Zhao et al., 2018; Ong et al., 2022). However, the complex marine environments such as fog, spray, waves, and most notably, frequent inter-vessel occlusion which is highlighted as a primary challenge in specialized benchmarks like MVDD13 (Wang et al., 2024b), can severely degrade image quality and lead to blurred or even completely obscured target features, threats severe challenges to vision-based marine vessel detection (Kortylewski et al., 2020, 2021). Occlusion is especially common in congested waterways and ports, substantially compromising the accuracy and reliability of existing detection models. Therefore, developing a ship detection method that maintains high accuracy under occlusion while meeting the stringent real-time and low-power constraints of embedded systems onboard unmanned platforms is crucial for advancing intelligent ocean observation and risk mitigation technologies.

The human visual system is capable of inferring the properties of objects based on the contours present in a scene, even when local information of the objects is occluded or lost (Rensink and Enns, 1998). However, it remains challenging for deep learning-based computer vision systems to effectively detect occluded objects (Sun et al., 2022). To address the occlusion problem, data augmentation methods such as Mosaic (Zeng et al., 2022), Cutout (Hinton et al., 2012) and CutMix (Yun et al., 2019), etc., have been proposed to only alleviate occlusion issues in certain extent. Some studies have designed specialized attention modules and contextual fusion mechanisms (Hu et al., 2020; Ranftl et al., 2021; Liu et al., 2021b; Hou et al., 2021) to improve the recognition of occluded targets. Occlusion can be regarded as a type of deformation problem. By extracting invariant features of ships, such as the gradient direction histogram feature based on the co-occurrence matrix (Kawahara et al., 2012) and constrained subspaces (Maki et al., 2002), local occlusion of ships can be addressed. However, these feature vectors are not universally applicable to all types of ships. Tang et al. (2022) combined occlusion detection strategies with matching algorithms to achieve continuous tracking after occlusion. However, it is not suitable for detecting occluded targets in static images. Recently, Wang et al. (2024a) proposed an attention-aware ship occlusion detection method and named AodeMar, to tackle the problem of feature confusion in occluded regions of ship targets. Specifically, a position enhancement module was constructed based on residual connections and coordinate attention (Wu et al., 2022). By fusing information from the horizontal and vertical spatial directions in the channel dimension, this module explicitly models interactions among positional features, thereby capturing long-range dependencies and strengthening feature representation for partially occluded targets. Additionally, a multi-scale feature semantic association method was proposed based on spatial pyramid pooling and sliding window self-attention encoders (Liu et al., 2021b). By sliding windows, this method establishes interactions between multi-scale target features within and across windows, enhancing the model's feature discrimination ability at both global and local levels. These enhancements typically further increase model complexity, making real-time inference on edge

computing devices. Consequently, a noticeable gap exists between detection accuracy, occlusion robustness and deploy ability on embedded systems. This gap is evident even in advanced models designed for complex, multi-source maritime imagery (e.g., optical, SAR), which often prioritize accuracy at the expense of computational efficiency, hindering their deployments (Wu et al., 2025).

Lightweight network designs offer pathways toward efficiency. Current research in this domain mainly focuses on two aspects. The first involves designing lightweight architectural modules by refining convolution operations to reduce parameter counts. For instance, MobileNet series models decompose traditional convolutions into depthwise and pointwise ones, incorporating width and resolution multipliers to compress channel dimensions and input resolution, thereby significantly lowering parameter (Howard et al., 2017; Sandler et al., 2018; Howard et al., 2020). However, the extensive use of depthwise convolutions can increase computational load. To mitigate this, ShuffleNet introduced group pointwise convolutions and channel shuffle operations to promote cross-group feature interaction, enhancing representational capacity while maintaining efficiency (Zhang et al., 2018; Ma et al., 2018). EfficientNets achieve a balanced scaling of network width, depth, and resolution to optimize the accuracy-efficiency trade-off (Tan and Le, 2019, 2021). More recently, GhostNet has gained attention for generating feature maps through cost-effective linear transformations, effectively revealing intrinsic features with reduced computational cost (Han et al., 2020). This design has been successfully applied in maritime contexts, such as in SAR image ship detection, to create compact models (Xiang et al., 2024). In maritime-specific applications, researchers have integrated depthwise separable convolution (DSC) (Chollet, 2017) into YOLOv4-based detectors to accelerate inference (Liu et al., 2021a), while others have combined ShuffleNet2 backbones with SE attention modules and DSC-enhanced necks to achieve competitive accuracy on datasets such as SMD (Yang et al., 2022). A key realization in this field is that parameter count alone does not linearly correlate with practical inference speed. Consequently, recent efforts increasingly emphasize computational complexity, measured in FLOPs, as a more direct indicator of on-device performance (Molchanov et al., 2016). For example, MobileNet-SSD and its variants demonstrate that coupling lightweight backbones with efficient detection heads can yield favorable speed-accuracy profiles (Howard et al., 2017; Sandler et al., 2018). Nevertheless, these gains in efficiency often come at the cost of degraded feature extraction capability, particularly in challenging visual scenarios. The second research thrust adopts model compression techniques to prune redundant network structures. To strike a balance between detection speed and accuracy, scholars have explored various compression strategies, including precision quantization, structured pruning (Li et al., 2016), and knowledge distillation (Wang et al., 2019) (Gao et al., 2021; Deng et al., 2020). However, aggressive compression, as seen in YOLO-LITE (Huang et al., 2018), can lead to severe accuracy degradation. Weight-based pruning algorithms, which remove neurons with relatively small contributions, offer a more granular approach to maintaining performance while reducing model size (Han et al., 2015).

The pursuit of higher mAP and lower FLOPs on server GPUs does not translate directly to embedded systems. Achieving a viable triple balance among detection accuracy, processing speed, and robustness against environmental variations on constrained edge devices poses a significant, often unmet, challenge for existing methods. Driven by the need for real-time perception on USVs, embedded vision platforms have motivated the development of lightweight detection networks that reconcile throughput with deployability. Notable instances include eWaSR, an embedded-ready maritime obstacle segmentation network that maintains robust accuracy while achieving real-time execution on platforms (Tersek et al., 2023). Similarly, pruned and quantized YOLO variants have been widely adopted for ship detection, offering substantial reductions in parameters and FLOPs (Adarsh et al., 2020; Wang et al., 2021; Haijoub et al., 2024; Li and Wang, 2025). To realize the full potential of these algorithms on edge hardware, dedicated optimization tools (e.g., TensorRT) and quantization strategies have proven essential for achieving practical throughput and energy efficiency (Martin-Salinas et al., 2025). Further innovations incorporate efficient feature-fusion mechanisms and cascaded detection pipelines to accelerate inference on edge devices (Liu et al., 2022; Sankaran et al., 2023).

In summary, current approaches to the combined challenge of occluded vessel detection for embedded maritime platforms lack dedicated modeling of occlusion patterns intrinsic to the maritime environment, cannot simultaneously satisfy the competing demands of high fidelity, low latency, and operational resilience on edge devices, and lack conclusive proof of viability in real-world settings. To address these limitations, this paper introduces eAodeMar, an algorithm-hardware co-designed framework that is rigorously validated through real-world deployments. Specially, key contributions are summarized as follows:

1. A lightweight and embedded-ready detection network eAodeMar is proposed for embedded maritime platforms. By systematically integrating Ghost convolution modules into both backbone and feature fusion stages, the architecture achieves a significant reduction in parameters and FLOPs while retaining the capacity to recognize partially visible ships in cluttered maritime scenes.
2. A complete algorithm-to-deployment co-design pipeline is established to bridge the gap between algorithmic efficiency and practical hardware execution. The pipeline includes structural lightweighting, TensorRT-based graph optimization, mixed-precision quantization, and tailored deployment on the NVIDIA Jetson Xavier NX platform, ensuring sustained real-time performance under real-world power and memory constraints.
3. Extensive evaluations on the public MVDD13 dataset and real-world maritime video sequences are conducted. The results demonstrate that eAodeMar effectively navigates the accuracy-efficiency trade-off, achieving a significant improvement in inference speed ($> 40\%$) with only a minimal detection accuracy drop, thereby validating its practicality for real-time onboard perception in occlusion-prone maritime scenarios.

The remainder of this paper is organized as follows. In Section 2, the proposed eAodeMar architecture and its lightweight design are presented. The embedded deployment of eAodeMar is detailed in Section 3. The experimental setup, ablation studies, and comparison results and corresponding analysis are provided in Section 4. Finally, conclusion and future work are drawn in Section 5.

2 The eAodeMar architecture

2.1 Architecture analysis

The factors contributing to the model's complexity are first analyzed in this section to guide the subsequent lightweight design. Two key metrics for evaluating model complexity are parameter count (Lecun et al., 1998, 2015) and FLOPs (Molchanov et al., 2016; Howard et al., 2017), which directly reflect the model's expressive capacity and its practical deployment feasibility (Cheng et al., 2017).

In our earlier work, we have introduced a position enhancement module named RCAC3 and a multi-scale semantic association module named SP-STR, which have been used to replace the original C3 modules in the high-level layers of the backbone and neck networks, respectively. To assess their impact on model complexity, we first analyzed the changes in parameter count resulting from these replacements. As shown in Table 1, both modules increase the parameter count compared to their corresponding C3 counterparts. The RCAC3 module in the backbone network introduced a relatively modest increase, whereas the SP-STR module in the neck network led to a more significant expansion, particularly in its first layer, where the parameter count nearly doubled.

As shown in Table 2, a detailed analysis of the computational and performance impacts caused by replacing the two modules reveals several key findings. Compared to the original AodeMar model, replacing the RCAC3 module with a standard C3 module reduces the total parameter count by 0.81%. This modification leads to a reduction in detection accuracy, with mAP@.5 and mAP@[.5:.95] decreasing by 0.59% (0.56) and 1.90% (1.57), respectively, while the change in computational load remains negligible. Notably, however, this replacement results in a significant 33.80% improvement in inference speed (FPS). When the SP-STR module

TABLE 1 Computation analysis of module parameters.

Component	Module	Module parameter	Parameter variation
Backbone network	RCAC3	1189400	↑0.56%
	C3	1182720	
Neck network	SP-STR	725508	↑100.43%
	C3	361984	
	SP-STR	1974792	↑66.97%
	C3	1182720	

“↑” denotes increase trend.

TABLE 2 The impact of RCAC3 and SP-STR on the AodeMar's computational load, detection accuracy, and speed.

Metrics	AodeMar	–RCAC3	–SP-STR
mAP@.5/%	95.43	94.87 (↓0.56)	95.23 (↓0.20)
mAP@[.5:.95]/%	82.57	81.00 (↓1.57)	82.43 (↓0.14)
Total parameters	8282502	8275822 (↓0.81%)	7126906 (↓13.95%)
FLOPs/G	67.40	67.40	16.30
FPS	61.73	82.59	68.97

“↓” denotes decrease trend, and “–” denotes that the module is replaced by C3 convolution.

in the neck network is replaced with its C3 counterpart, the model exhibits a more pronounced reduction in parameter count down by 13.95%. The corresponding declines in detection accuracy are relatively modest, with mAP@.5 and mAP@[.5:.95] dropping by only 0.21% (0.20) and 0.17% (0.14), respectively. More strikingly, computational load is dramatically reduced to approximately one-fourth of the original, accompanied by an 11.72% increase in FPS.

These results collectively demonstrate that both RCAC3 and SP-STR modules achieve marginal improvements in detection accuracy at the cost of substantial memory consumption and reduced inference efficiency. The SP-STR module, in particular, exerts a more significant influence on overall model performance. Consequently, optimizing these two modules is essential to enable lightweight, efficient deployment without compromising practical usability.

2.2 Lightweight occluded vessel detection network eAodeMar

Based on the analysis in the previous section, this section focuses on the lightweight design of the RCAC3 and SP-STR modules. In convolutional neural networks, input images are

progressively transformed through a series of operations such as convolution and pooling into output feature maps. Conventional convolution layers, however, often incur substantial computational overhead. Moreover, the inherent properties of original convolution tend to generate a considerable number of redundant or highly similar feature maps during training, which further elevates computational complexity and model size without proportionally improving representational capacity.

2.2.1 Lightweight module design based on ghost convolution

Ghost convolution (Han et al., 2020), introduced by Huawei Noah's Ark Lab, provides an efficient alternative to original convolution by substantially reducing computational complexity while maintaining comparable representational capacity. To quantitatively demonstrate the efficiency advantage of Ghost convolution over original convolution, we analyze both parameter count and FLOPs. As shown in Figure 1, let the input tensor be $\mathcal{X} \in \mathbb{R}^{h \times w \times c}$, where h , w and c denote its height, width and number of input channels, respectively.

For original convolution, applying an operation F with n convolution kernels of size $k_h \times k_w$ can generate n feature maps. Ignoring the bias term, the output can be given by

$$\mathcal{Y} = F(\mathcal{X})$$

where $\mathcal{Y} \in \mathbb{R}^{\tilde{h} \times \tilde{w} \times n}$, $\tilde{h}$ and $\tilde{w}$ denote height and width of the output feature maps, respectively. The number of parameters and FLOPs can be expressed by

$$P_o = c \times n \times k_h \times k_w \quad (1)$$

$$F_o = \tilde{h} \times \tilde{w} \times c \times n \times k_h \times k_w \quad (2)$$

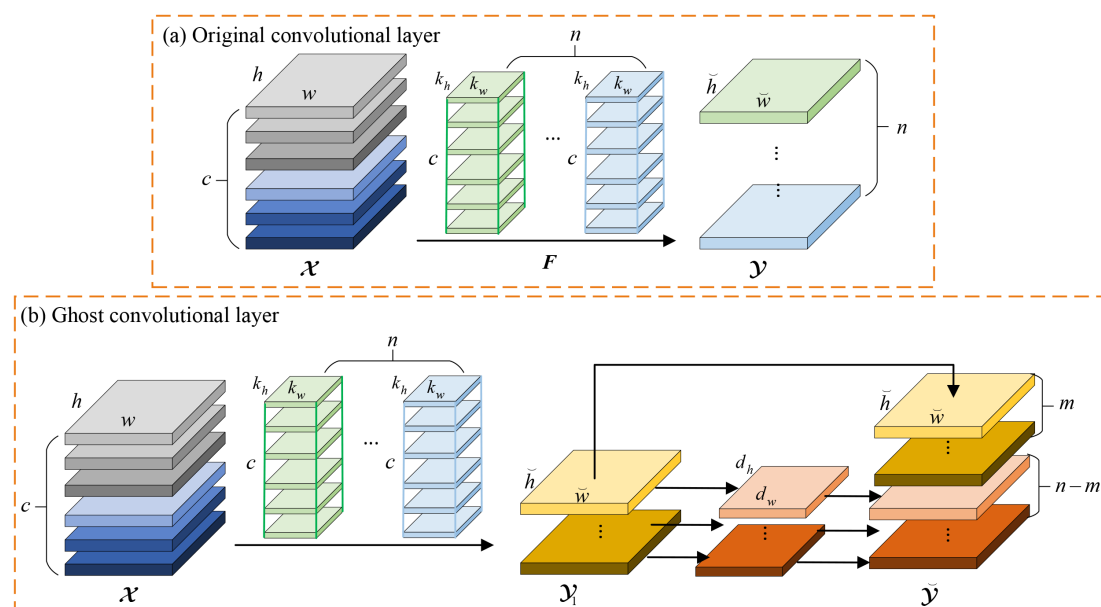


FIGURE 1

Structural diagrams contrasting the (a) Original convolution and (b) Ghost convolution, highlighting the reduction in redundant operations and parameters.

where P_o is the number of parameters, F_o denotes FLOPs, c and n represent the number of channels of input and output feature maps, respectively.

In Ghost convolution, only m intrinsic feature maps are generated via original convolution, where $m \ll n$, and the remaining $n - m$ “ghost” feature maps are produced by applying low-cost depthwise convolutions with small kernels. Let the kernel size of the linear transformation be denoted as $d_h \times d_w$ (typically $d_h, d_w \leq k_h, k_w$). The parameter count and FLOPs of Ghost convolution can be computed by

$$P_g = c \times m \times k_h \times k_w + m \times (n - m) \times d_h \times d_w \quad (3)$$

$$F_g = \tilde{h} \times \tilde{w} \times [c \times m \times k_h \times k_w + m \times (n - m) \times d_h \times d_w] \quad (4)$$

where P_g and F_g denote parameters and FLOPs of the Ghost convolution, respectively. $\tilde{h}$ and $\tilde{w}$ represent height and width of the output feature maps, respectively.

In this context, the theoretical compression ratio r_s in terms of parameters and FLOPs can be approximated as:

$$r_s \approx \frac{m}{n} + \frac{(n - m) \times d_h \times d_w}{n \times k_h \times k_w}. \quad (5)$$

In practice, setting $m \approx n/2$ and using small linear kernels (e.g., $d_h = d_w = 3$) typically yields a reduction in both parameters and FLOPs by approximately half compared to original convolution, without significant loss in accuracy.

This intrinsic efficiency makes Ghost convolution particularly suitable for deploying detection models on resource-constrained embedded platforms, where minimizing computational overhead and memory footprint is paramount. By replacing the original convolutions in the RCAC3 and SP-STR modules with Ghost convolutions, a markedly compact architecture can be obtained.

Remark 1. Note that $m \approx n/2$ adheres to the efficient design established in GhostNet Han et al. (2020). Moreover, this design leads to a final model that achieves a significant reduction in parameters and FLOPs while preserving the accuracy necessary for occluded vessel detection, as will be demonstrated in the following Results and Analysis section.

2.2.2 Network architecture of eAodeMar

To simultaneously improve detection speed while maintaining the original modules' performance as much as possible, we employ Ghost convolution (Han et al., 2020) for lightweight redesign, balancing efficiency with representational capacity. For clarity, the two resulting lightweight modules are denoted as G-RCAC3 and G-SP-STR, respectively, and the overall optimized and efficient model is named eAodeMar. The complete architecture is illustrated in Figure 2.

As shown in Table 3, the lightweight redesign leads to a significant reduction in parameter count for both modules. Specifically, G-RCAC3 achieves a 21.65% reduction in parameters compared to the original RCAC3 module. Meanwhile, G-SP-STR, which processes the smaller-scale 512-channel feature maps in the neck network, undergoes a 13.04% parameter reduction. These reductions directly contribute to lower memory footprint and

enhanced inference efficiency, making the model better suited for deployment on resource-constrained embedded platforms while preserving detection accuracy.

From Equations 1 and 3, it can be derived that the reduction in model parameters is intrinsically linked to both the kernel size and the channel dimensions. When the number of input channels is substantial, this parameter-saving mechanism becomes particularly pronounced, highlighting the efficiency advantage of the adopted lightweight design.

As shown in Table 3, the variation in computational load reflects the impact of replacing individual modules within the overall network. It is noteworthy that, following the substitution with lightweight counterparts, the backbone network exhibits a relatively minor increase in computational overhead. This observation aligns with the mathematical relationships expressed in Equations 2 and 4, which indicate that reductions in model FLOPs are also strongly influenced by kernel dimensions and channel configurations.

Remark 2. Note that Equation 5 considers only convolutional operations and does not incorporate computational contributions from bias terms, activation functions such as SiLU, or other layer-wise operations. Therefore, the theoretical compression ratio presented here denotes idealized upper limits. In practical deployment scenarios, the actual achievable reductions tend to be less pronounced.

3 Embedded deployment pipeline of lightweight eAodeMar

3.1 Deployment platform

As shown in Figure 3, the deployment is conducted on the NVIDIA Jetson Xavier NX embedded GPU module, a compact platform (70 mm × 45 mm) supporting configurable power modes up to 20 W and delivering peak performance of 21 TOPS. Key specifications are listed in Table 4. The module integrates a 6-core NVIDIA Carmel ARM CPU and a 384-core Volta GPU with 48 Tensor cores, paired with 8 GB of shared memory. It also includes dedicated accelerators for deep learning and vision tasks, along with programmable vision engines. Connectivity features comprise PCIe 3.0/4.0 interfaces for storage, a 16-lane MIPI CSI-2 camera interface, DP/HDMI outputs, and low-speed interfaces (IIC, SPI, UART and IIS) for peripheral integration, making it well-suited for real-time maritime vision applications.

3.2 Deployment process

The deployment of the lightweight eAodeMar model onto the Jetson Xavier NX embedded GPU encompasses a structured optimization and acceleration pipeline designed to ensure efficient real-time inference. The overall workflow, executed within an Ubuntu 18.04 environment updated with JetPack 4.6, involves environment configuration, model acceleration using TensorRT, and the final on-device inference implementation.

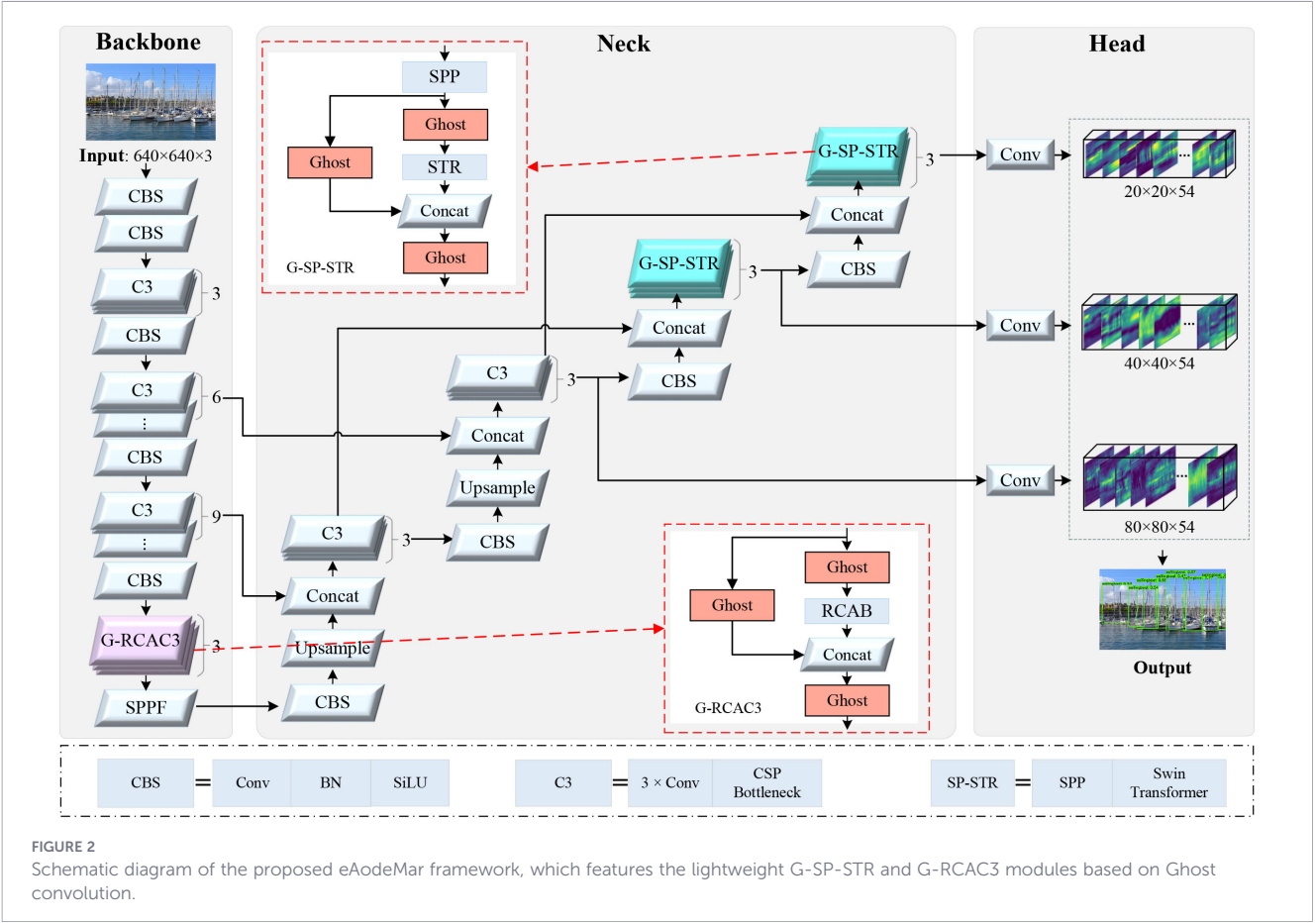


FIGURE 2 Schematic diagram of the proposed eAodeMar framework, which features the lightweight G-SP-STR and G-RCAC3 modules based on Ghost convolution.

(a) TensorRT-based Model Acceleration.

Acceleration is primarily achieved through parameter quantization and graph-level structural optimization. First, network parameters are quantized from FP32 to FP16 precision, reducing memory bandwidth and computational demand while maintaining acceptable accuracy. Subsequently, TensorRT performs a comprehensive graph-level refactoring that merges compatible layers both vertically and horizontally to minimize the complexity of the execution graph.

As illustrated in Figure 4, each layer in a typical inference graph invokes a separate CUDA kernel. Vertical fusion combines operations such as convolution, bias addition, and SiLU activation into a single C-B-S (Convolution-Bias-SiLU) module, thereby reducing the number of kernel launches from three to one. Horizontal fusion is applied to C-B-S layers that share the same

input tensor and perform identical operations, further streamlining the computational graph. These optimizations collectively decrease latency and improve throughput, enabling the model to meet real-time performance requirements on the embedded platform.

(b) Implementation of the TensorRT Inference Framework.

The inference acceleration framework is realized through a sequential three-stage workflow: model conversion, engine construction, and runtime execution. First, the PyTorch-trained model (.pt) is exported to the Open Neural Network Exchange (ONNX) format (.onnx) to ensure framework interoperability. The ONNX file is then transferred to the Jetson Xavier NX, where a TensorRT builder parses the model graph. Within the CUDA environment, the builder applies kernel auto-selection, layer fusion, and FP16 quantization to generate a highly optimized execution plan. This plan is serialized into a portable TensorRT

TABLE 3 Module-wise comparison of parameters and computational complexity before and after lightweight optimization.

Component	Module	Channels of input, output	FLOPs/G	Variation	Parameter	Variation
Backbone network	G-RCAC3	512,512	67.20	↓0.30%	0.93	↓21.65%
	RCAC3		67.40		1.19	
Neck network	G-SP-STR	256,256	66.90 67.40	↓0.74%	0.66	↓8.72%
	SP-STR				0.73	
	G-SP-STR	512,512			1.72	↓13.04%
	SP-STR				1.97	

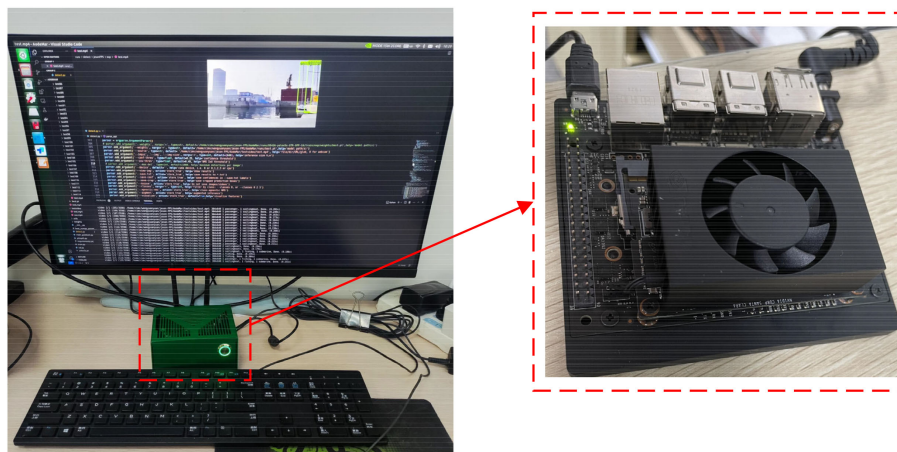


FIGURE 3
Experimental devices and Jetson Xavier NX GPU.

engine file (.engine). During inference, the engine is deserialized to restore the complete network definition, trained parameters, and pre-allocated activation buffers. Forward passes are executed with minimal overhead, producing output tensors that encode detected ship proposals. Finally, these outputs are decoded and processed through Non-Maximum Suppression (NMS) to eliminate redundant bounding boxes, completing the real-time ship detection for maritime video streams.

4 Results and analysis

4.1 Implementation details and datasets

The experimental configuration is detailed in Table 5. To ensure a fair comparison, all models are trained, validated, and tested using an input resolution of 640×640 pixels. During training, a batch size of 16 was employed, and models were optimized for 300 epochs until convergence. Inference speed is reported in FPS, which accounts for the complete processing pipeline, including image pre-processing, forward pass, and NMS, rather than the forward pass alone.

All experiments are conducted on a publicly available maritime vessel detection dataset, MVDD13¹ (Wang et al., 2024b). This dataset comprises 13 ship categories with precise bounding-box annotations. For all models, 25,541, 2,838 and 7,095 images are used for training, validation and testing, respectively. To further assess the practical applicability of the deployed model in real-world maritime scenarios, the most challenging on-board video sequences of SMD² (Moosbauer et al., 2019) dataset and the video data captured by our USV operating in the Linghai campus wharf of Dalian. Sample frames are shown in Figure 5.

¹ <https://github.com/yyuanwang1010/MVDD13>.

² <https://sites.google.com/site/dilipprasad/home/singapore-maritime-dataset>.

4.2 Evaluation metrics

Given an IoU threshold, the Precision and Recall can be determined accordingly. To this end, the AP for each category can be calculated by

$$AP = \int_0^1 \Pr(\text{Re}) d_{\text{Re}}$$

where Pr and Re denote the Precision and Recall, respectively.

Accordingly, the mean average precision (mAP) for all categories in specific IoU thresholds, i.e., mAP@.5 and mAP@[.5:.95], can be obtained. In addition, the FPS³ can be calculated directly by the number of frames divided by time cost.

4.3 Ablation Studies

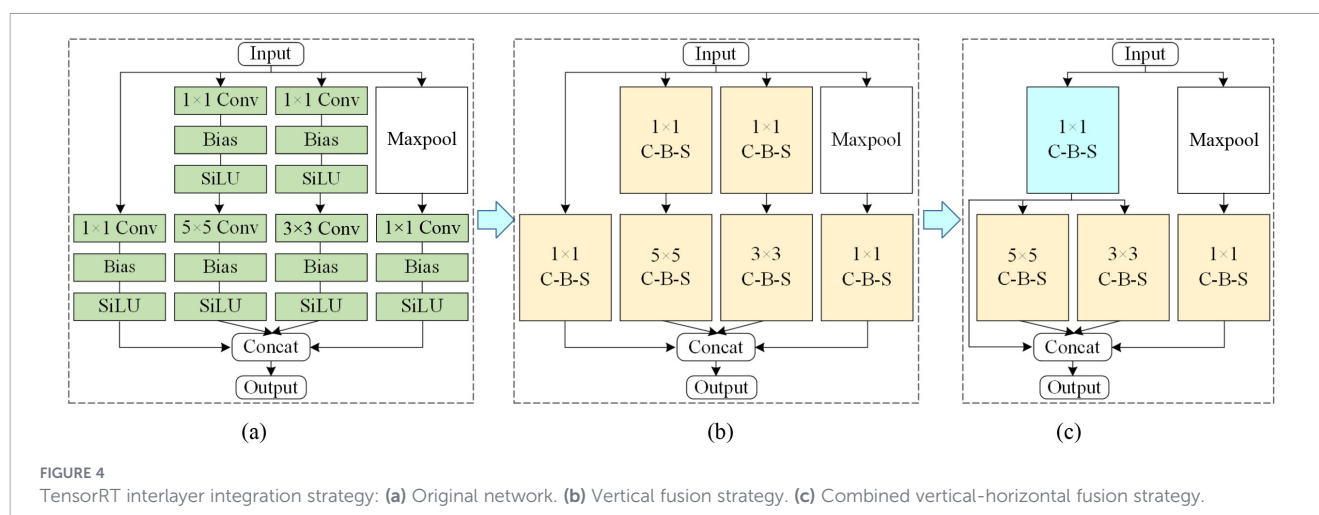
4.3.1 Influence of the Two Lightweight Modules on Model Performance

The results are shown in Table 6. Compared with the original AodeMar model, the variant employing only the lightweight G-

TABLE 4 Hardware parameters of Jetson Xavier NX.

Parameter	Detailed description
AI Performance	21TOPS (INT8) or 6.8TFLOPS (FP16)
GPU	384 CUDA cores and 48 Tensor Cores
CPU	6-core NVIDIA Carmel ARM [®] v8.2 64-bit CPU
Memory	8 GB 128-bit LPDDR4x
Storage	16 GB eMMC 5.1 flash memory
Power	10W/15W/20W
Linux	Ubuntu 18.04
TensorRT	TensorRT 7.1.3

³ <https://mmdetection.readthedocs.io/zhCN/latest/userguides/usefultools.html?highlight=FPSfps>.



RCAC3 module exhibits a reduction in mAP@.5 of 3.14%, accompanied by moderate decreases in both parameter count and FLOPs, while achieving a 32.40% improvement in FPS. Similarly, the model using only the G-SP-STR module shows a smaller accuracy drop of 0.62% in mAP@.5, together with reduced model complexity, and attains a 35.24% gain in FPS.

When both lightweight modules are integrated simultaneously, the resulting eAodeMar model achieves the most favorable balance across all evaluated metrics. Specifically, it maintains a nearly identical detection accuracy, with only a 0.42% decrease in mAP@.5, while substantially reducing computational complexity. Most notably, the inference speed is increased by nearly 26 FPS, corresponding to a 42.12% improvement relative to the baseline. These results validate the effectiveness of the proposed Ghost convolution-based lightweight design in enhancing real-time performance with minimal compromise in detection accuracy.

4.3.2 Influence of lightweight backbones on model performance

To systematically evaluate the impact of lightweight backbone networks, we conducted an ablation study comparing several well-established lightweight architectures: GhostNet (Han et al., 2020), ShuffleNetv2 (Ma et al., 2018), MobileNetv3 ep2020Searching, PP-LCNet (Cui et al., 2021), and EfficientNetv2 (Tan and Le, 2021).

The comparative results are presented in Table 7. Among all lightweight backbone variants, eAodeMar achieves the second-highest inference speed, surpassed only by A-PP-LCNet. However, its advantages in parameter count and FLOPs are less pronounced. The eAodeMar ranks second in parameter efficiency (after A-GhostNetv2) while exhibiting approximately 10% higher FLOPs. Notably, eAodeMar keeps a clear lead in detection accuracy over all lightweight backbone alternatives. Within the lightweight backbone group, A-PP-LCNet exhibits the lowest parameter count and FLOPs, resulting in the fastest inference speed. A-EfficientNet achieves the highest detection accuracy in both mAP@.5 and mAP@[.5:.95], closely followed by A-GhostNetv2. In terms of FPS, A-PP-LCNet ranks first, with A-EfficientNet in second place.

Compared to the original AodeMar model, all lightweight backbones substantially reduce parameter count and computational complexity (with the exception of MobileNetv3) and significantly improve FPS. For example, A-PP-LCNet increases FPS by 44.00%, while A-EfficientNet achieves a 27.05% improvement. However, replacing the backbone with any lightweight architecture consistently leads to a considerable drop in detection accuracy. The smallest accuracy degradation is observed with A-EfficientNet, which exhibits reductions of 2.97% in mAP@.5 and 0.97% in mAP@[.5:.95].

4.4 Comparison experimental results

4.4.1 Comparison with state-of-the-art lightweight models

To validate the superiority of the proposed eAodeMar model, we conduct a comparative evaluation against several mainstream lightweight object detectors, including YOLOv3-Tiny (Adarsh et al., 2020), YOLOv4-Tiny (Wang et al., 2021), YOLOv5-Lite, and YOLOv7-Tiny.

The comparison results are shown in Table 8. It can be observed that existing lightweight models generally exhibit lower parameter counts and FLOPs. In particular, YOLOv5-Lite uses only about 20% of the parameters and 5% of the FLOPs compared to AodeMar/

TABLE 5 The training environment configuration.

Parameter	Configuration
Linux	Ubuntu 18.04
CPU	Intel Core i7-8700K
GPU	GeForce RTX 2080Ti
CUDA	CUDA 10.2
cuDNN	cuDNN v8.3.2
Algorithm framework	PyTorch 1.7.1
Optimizer	SGD

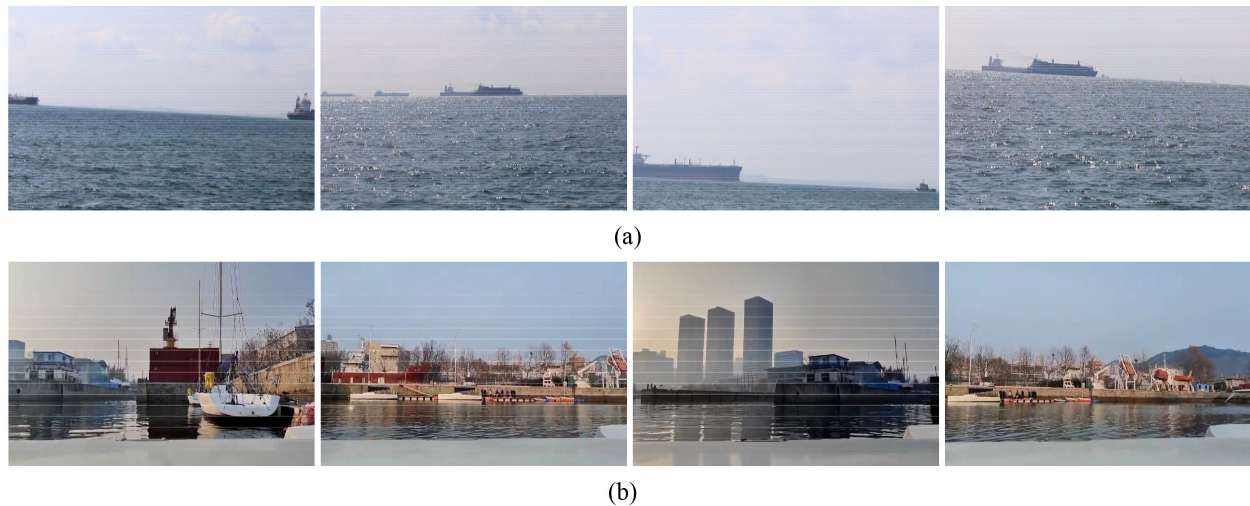


FIGURE 5

Example images from (a) SMD onboard video and (b) Linghai campus wharf video used for evaluation, featuring challenging scenarios with varied obstacles.

eAodeMar. However, its inference speed exceeds eAodeMar by only 12.28 FPS (a 14% improvement), while suffering a significant degradation in detection accuracy, an 8% drop in mAP@[.5:.95] and a 14.3% drop in mAP@.5.

Furthermore, the results reveal that parameter count and FLOPs are not strictly proportional to actual inference speed. Although eAodeMar has relatively higher parameter and FLOP values, it still achieves faster inference than the widely adopted YOLOv7-Tiny model. These comparisons demonstrate that eAodeMar maintains a clear advantage in detection accuracy while exhibiting competitive inference efficiency, with notable potential for further speed optimization.

4.4.2 Performance comparison after TensorRT acceleration

For convenience, the TensorRT-accelerated model is denoted as tAodeMar. In this study, both the baseline AodeMar and the lightweight eAodeMar are initially deployed on the Jetson Xavier NX platform using the PyTorch framework for reference performance evaluation.

As shown in Table 9, we compare the stage-wise detection accuracy, inference speed, and time consumption of AodeMar, eAodeMar and tAodeMar models on the MVDD13 image test set. Compared to the original AodeMar running on the embedded platform, eAodeMar reduces the time of all three phases (pre-processing, inference, NMS), raising an overall speed improvement of 4.5%. Following TensorRT acceleration, the optimized tAodeMar model achieves a 61.56% reduction in inference time, effectively doubling the overall detection speed to 37.45 FPS. Although a marginal decrease of 0.23% in mAP@0.5 is observed, this minimal accuracy loss strongly validates the practical efficacy of the tAodeMar model, confirming that significant acceleration can be attained with negligible degradation in detection performance.

To thoroughly evaluate the real-time performance of these models under practical deployment conditions, we conducted inference tests on two distinct maritime video sequences: (1) a real-world video captured by the our USV operating in the Linghai campus wharf of Dalian, and (2) the publicly available SMD onboard ship video.

As summarized in Table 10, the AodeMar, eAodeMar, and tAodeMar models achieved inference speeds of 3.60, 5.76 and 18.31 FPS, respectively, on the real-world wharf video, and 6.93, 7.33 and 28.57 FPS on the SMD video. These results demonstrate a marked improvement in detection speed following TensorRT acceleration. While the tAodeMar model attains 28.57 FPS on the SMD video, clearly satisfying the common real-time threshold of ≥ 20 FPS, its performance on the more complex real-world video (18.31 FPS) slightly falls below this threshold. As illustrated in Figure 5, the discrepancy is largely attributable to differences in scene complexity, the wharf environment contains richer background clutter and finer details compared to the open-sea scenes in the SMD video. Such complexity tends to generate a higher number of candidate proposals during detection, thereby increasing computational overhead and reducing frame-rate. In total, the TensorRT-accelerated model (tAodeMar) achieves a substantial boost in inference speed while confining the loss in detection accuracy to an acceptable margin. These outcomes validate both the effectiveness of the proposed lightweight-acceleration pipeline and its practical applicability in real-world maritime perception scenarios.

To qualitatively compare the visual detection results before and after acceleration, four representative frames are selected from the real-world video, covering both head-light (row 1) and back-light (rows 2-4) illumination scenarios, as well as challenging scenarios involving occlusion and partially visible objects. The detection results are shown in Figure 6. In the head-light occlusion scene (row 1), both models successfully detect two heavily occluded vessels, indicating that the lightweight design retains the ability to recognize targets

TABLE 6 Performance comparisons between AodeMar and its lightweight models.

Metrics	AodeMar	+G-RCAC3	+G-SP-STR	eAodeMar
mAP@.5/%	95.43	95.13	94.84	95.03
Total parameters/M	8.28	8.02	7.96	7.70
FLOPs/G	67.40	67.20	66.90	66.80
FPS	61.73	81.30	83.33	87.72

“+” stands for C3 convolution is replaced by the module.

TABLE 7 Comparisons among eAodeMar, AodeMar and models with lightweight backbones.

Model	Parameters/M	FLOPs/G	mAP@.5/%	mAP@[.5:.95]/%	FPS
A-GhostNetv2	7.46	59.4	92.57	81.73	69.93
A-ShuffleNetv2	5.81	57.6	90.77	78.07	75.19
A-MobileNetv3	6.15	57.9	91.79	79.87	51.68
A-PP-LCNet	5.63	57.6	91.64	79.44	88.89
A-EfficientNet	6.18	59.0	92.60	81.77	78.43
AodeMar	8.28	67.4	95.43	82.57	61.73
eAodeMar	7.70	66.8	95.03	84.53	87.72

“A-X”denotes AodeMar with backbone replaced by X.

TABLE 8 Quantitative comparisons with typical lightweight models.

Model	Parameters/M	FLOPs/G	mAP@.5/%	mAP@[.5:.95]/%	FPS
YOLOv3-Tiny	4.91	5.61	78.44	43.52	200.04
YOLOv4-Tiny	5.90	16.22	83.01	52.36	179.15
YOLOv5-Lite	1.56	3.80	87.34	70.23	100.00
YOLOv7-Tiny	6.23	13.86	90.56	78.46	83.86
AodeMar	8.28	67.40	95.43	82.57	61.73
eAodeMar	7.70	66.8	95.03	84.53	87.72

TABLE 9 Performance comparison of models before and after deployment.

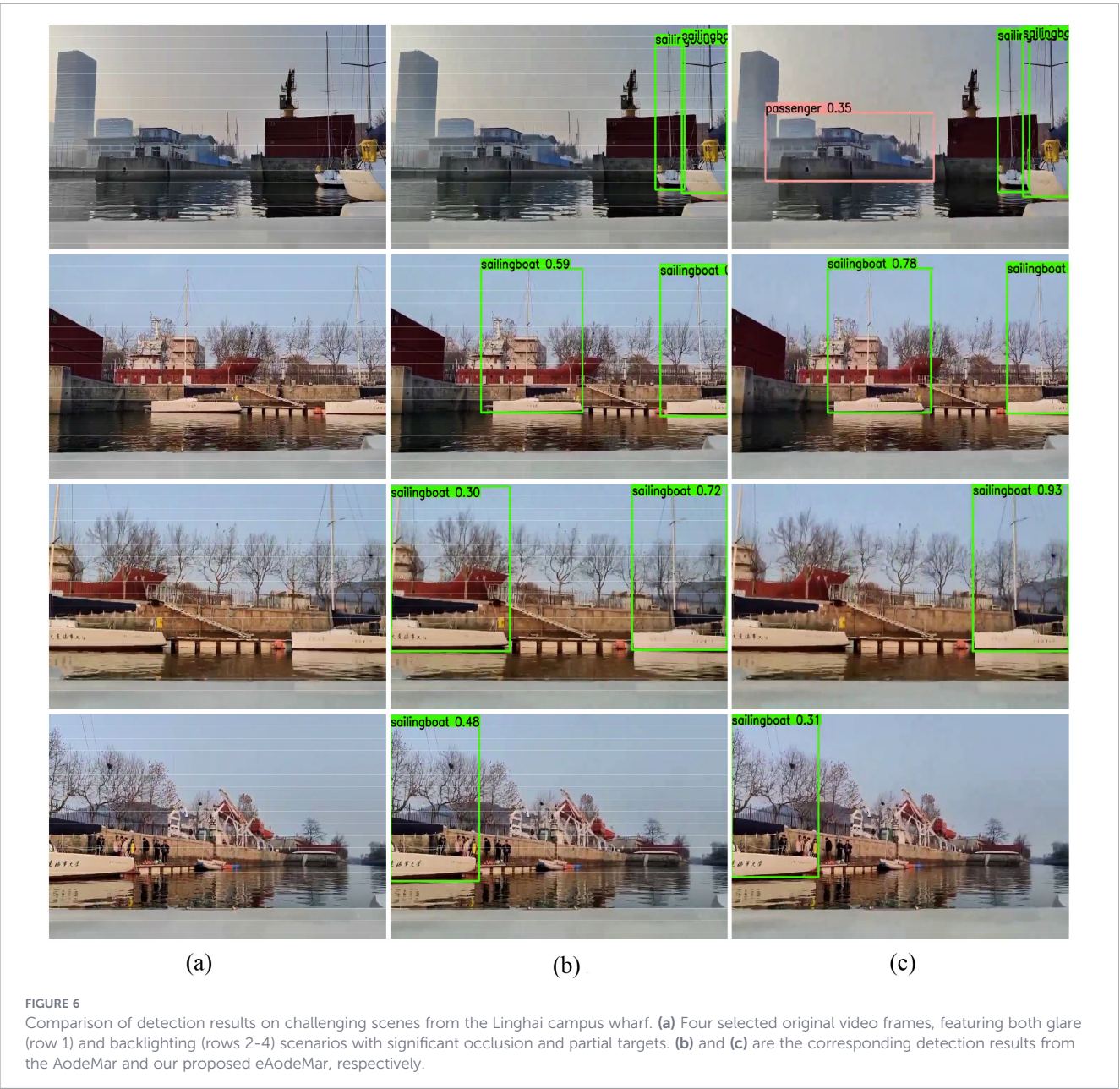
Model	mAP@.5/%	Pre-processing/ms	Inference/ms	NMS/ms	FPS (Jetson)
AodeMar (PyTorch)	95.40	1.7	60.2	3.4	15.31
eAodeMar (PyTorch)	95.09	1.5	58.0	3.0	16.00
tAodeMar (TensorRT-FP16)	94.87	1.4	22.3	3.0	37.45

under severe occlusion. However, due to the low illumination associated with the head-light setting, a false positive occurs, i.e., a building with windows extending over the water on the left pier is incorrectly classified as a passenger ship. This is attributed to the reduced model complexity of the lightweight design, which slightly weakens feature extraction capability, leading to occasional false and missed detections. For the two clearly visible targets in row 2 and the

partially visible one on the right in row 3, both models achieve correct identification and localization with high confidence. In contrast, for the partially visible sailboats on the left in rows 3 and 4, despite their relatively large area within the frame, both models yield low confidence scores, while the eAodeMar model even misses the target entirely. This observation indicates that detection performance is not directly proportional to the objects image size.

TABLE 10 Comparison of detection speed before and after acceleration on the actual collected and SMD video streams.

Video data	Model	Pre-processing/ms	Inference/ms	NMS/ms	FPS (Jetson)
Actual collected video	AodeMar (PyTorch)	1.7	60.2	3.4	15.31
	eAodeMar (PyTorch)	1.5	58.0	3.0	16.00
	tAodeMar (TensorRT-FP16)	1.4	22.3	3.0	37.45
SMD	AodeMar (PyTorch)	1.7	60.2	3.4	15.31
	eAodeMar (PyTorch)	1.5	58.0	3.0	16.00
	tAodeMar (TensorRT-FP16)	1.4	22.3	3.0	37.45



Instead, the detection outcome depends more strongly on the completeness and distinctiveness of salient structural features (e.g., sails or masts), a behavior that aligns well with human visual recognition mechanisms.

In total, the eAodeMar model illustrates the capability to accurately localize and rapidly identify vessels in complex maritime environments. While a modest trade-off in detection accuracy is observed, the model achieves a substantial enhancement in inference speed. For real-world applications that necessitate a balanced compromise between detection performance and processing efficiency, eAodeMar presents a highly viable and practical solution.

5 Conclusion and future work

To enable the practical deployment of an occluded ship detection model on embedded GPUs, this paper proposes eAodeMar, a lightweight ship detection model constructed using Ghost convolution-based modules. Compared to the original AodeMar model, eAodeMar achieves a significant improvement in detection speed with only a marginal decrease in detection accuracy. Furthermore, the eAodeMar model was deployed on the Jetson Xavier NX embedded GPU, where TensorRT was employed for model optimization and acceleration. Performance was evaluated on the MVDD13 test set, video data collected in the Haitou harbor basin, and the SMD video dataset. The results show that the TensorRT-optimized model substantially reduces inference latency, leading to a remarkable increase in detection speed. Specifically, the optimized model achieves 37.45 FPS on the test set and 28.57 FPS on the SMD video, representing a significant improvement over both AodeMar and the non-accelerated eAodeMar. This successfully meets the initial deployment objective of enhancing detection speed while considerably lowering the hardware requirements of the model. Future work will focus on (1) advanced inference strategies like cascaded detection, adaptive mechanisms for complex scenarios; (2) lightweight attention and contextual modeling to better recognize partially visible targets; and (3) enhanced robustness under diverse, extreme maritime conditions, to build upon this efficient baseline for next-generation autonomous marine vision systems.

Data availability statement

The original contributions presented in the study are included in the article/supplementary material, further inquiries can be directed to the corresponding author/s.

Author contributions

YW: Data curation, Conceptualization, Validation, Methodology, Writing – review & editing, Investigation, Formal analysis, Writing – original draft, Supervision, Funding acquisition, Software. MW: Supervision, Writing – review & editing. JS: Writing – review & editing, Investigation, Funding acquisition. ZN: Conceptualization, Writing – review & editing, Investigation, Software. GL: Methodology, Software, Writing – original draft, Writing – review & editing, Data curation.

Funding

The author(s) declared that financial support was received for this work and/or its publication. This work is supported by the Scientific Research Foundations of Hainan Tropical Ocean University under Grants RHDRCZK202526 and RHDRCZK202402.

Conflict of interest

Author JS was employed by the company Weihai Guangtai Airport Equipment Co., Ltd.

The remaining author(s) declared that this work was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Generative AI statement

The author(s) declared that generative AI was not used in the creation of this manuscript.

Any alternative text (alt text) provided alongside figures in this article has been generated by Frontiers with the support of artificial intelligence and reasonable efforts have been made to ensure accuracy, including review by the authors wherever possible. If you identify any issues, please contact us.

Publisher's note

All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

References

- Adarsh, P., Rath, P., and Kumar, M. (2020). "YOLOv3-Tiny: Object detection and recognition using one stage improved model," in *2020 6th International Conference on Advanced Computing and Communication Systems*, (Coimbatore, India: IEEE). 687–694.
- Cheng, Y., Wang, D., Zhou, P., and Zhang, T. (2018). Model compression and acceleration for deep neural networks: The principles, progress, and challenges, *IEEE Signal Processing Magazine*, 35, 126–136. doi: 10.1109/MSP.2017.2765695
- Chollet, F. (2017). "Xception: Deep learning with depthwise separable convolutions," in *IEEE Conference on Computer Vision and Pattern Recognition*, (Honolulu, HI, USA: IEEE). 1800–1807.
- Cui, C., Gao, T., Wei, S., Du, Y., Guo, R., Dong, S., et al. (2021). "PP-LCNet: A lightweight CPU convolutional neural network," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, (Montreal, QC, Canada: IEEE). 356–365.
- Deng, L., Li, G., Han, S., Shi, L., and Xie, Y. (2020). Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proc. IEEE* 108, 485–532. doi: 10.1109/JPROC.2020.2976475
- Gao, H., Tian, Y., Xu, F., and Zhong, S. (2021). Survey of deep learning model compression and acceleration. *J. Software* 32, 1, 68–92. doi: 10.13328/j.cnki.jos.006096
- Gao, H., Wang, C., Niu, R., Fang, X., Chen, J., Sun, Y., et al. (2025). Driving risk assessment for intelligent vehicles based on entropy-informed graph neural networks and gaussian distributions. *IEEE Trans. Neural Networks Learn. Syst.* 36, 16478–16491. doi: 10.1109/TNNLS.2025.3569826
- Hajjoub, A., Hatim, A., Guerrero-Gonzalez, A., Arioua, M., and Chougali, K. (2024). Enhanced YOLOv8 ship detection empower unmanned surface vehicles for advanced maritime surveillance. *J. Imaging* 10, 303. doi: 10.3390/jimaging10120303
- Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., and Xu, C. (2020). "GhostNet: More features from cheap operations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (Seattle, WA, USA: IEEE). 1577–1586.
- Han, S., Pool, J., Tran, J., and Dally, W. J. (2015). "Learning both weights and connections for efficient neural networks," in *Proceedings of the 29th International Conference on Neural Information Processing Systems*, (Barcelona, Spain: Curran Associates, Inc). 1135–1143.
- Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. R. (2012). Improving neural networks by preventing co-adaptation of feature detectors. *Comput. Sci.* 3, 4, 212–223. doi: 10.9774/GLEAF.978-1-909493-38-4_2
- Hou, Q., Zhou, D., and Feng, J. (2021). "Coordinate attention for efficient mobile network design," in *IEEE Conference on Computer Vision and Pattern Recognition*, (Nashville, TN, USA: IEEE). 13708–13717. doi: 10.1109/CVPR46437.2021.01350
- Howard, A., Sandler, M., Chen, B., Wang, W., Chen, L. C., Tan, M., et al. (2020). "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, (Seoul, Korea (South): IEEE). 1314–1324.
- Howard, A., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *ArXiv*. [Preprint]. doi: 10.48550/arXiv.1704.04861
- Hu, J., Shen, L., Albanie, S., Sun, G., and Wu, E. (2020). Squeeze-and-excitation networks. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 2011–2023. doi: 10.1109/TPAMI.2019.2913372
- Huang, R., Pedoeem, J., and Chen, C. (2018). "YOLO-LITE: A real-time object detection algorithm optimized for non-GPU computers," in *2018 IEEE International Conference on Big Data*, (Seattle, WA, USA: IEEE). 2503–2510.
- Kawahara, T., Toda, S., Mikami, A., and Tanabe, M. (2012). "Automatic ship recognition robust against aspect angle changes and occlusions," in *Proceedings of IEEE National Radar Conference*, (Atlanta, GA, USA: IEEE). 864–869.
- Kortylewski, A., Liu, Q., Wang, A., Sun, Y., and Yuille, A. (2021). Compositional convolutional neural networks: A robust and interpretable model for object recognition under occlusion. *Int. J. Comput. Vision* 129, 736–760. doi: 10.1007/s11263-020-01401-3
- Kortylewski, A., Liu, Q., Wang, H., Zhang, Z., and Yuille, A. (2020). "Combining compositional models and deep networks for robust object classification under occlusion," in *Workshop on Applications of Computer Vision*, (Snowmass, CO, USA: IEEE). 1322–1330.
- Lecun, Y., Bengio, Y., and Hinton, G. E. (2015). Deep learning. *Nature* 521, 436–444. doi: 10.1038/nature14539
- Lecun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 2278–2324. doi: 10.1109/5.726791
- Li, H., Kadav, A., Durdanovic, I., Samet, H., and Graf, H. P. (2016). "Pruning filters for efficient convnets," in *Proceedings of the International Conference on Learning Representation*, (Hobart, Tasmania, Australia: Curran Associates, Inc). 24–26.
- Li, Y., and Wang, S. (2025). EGM-YOLOv8: A lightweight ship detection model with efficient feature fusion and attention mechanisms. *J. Mar. Sci. Eng.* 13, 757. doi: 10.3390/jmse13040757
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., et al. (2021b). "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE Int. Conf. Comput. Vis.*, (Montreal, Canada: IEEE). 9992–10002.
- Liu, J., Sun, Y., Wang, C., Hai, Z., and Li, J. (2022). Real-time ship detection in maritime environments using an efficient feature fusion network on embedded GPUs. *Ocean Eng.* 266, 113074. doi: 10.1016/j.oceaneng.2022.113074
- Liu, T., Pang, B., Zhang, L., Yang, W., and Sun, X. (2021a). Sea surface object detection algorithm based on YOLOv4 fused with reverse depthwise separable convolution (RDSC) for USV. *J. Mar. Sci. Eng.* 9, 753. doi: 10.3390/jmse9070753
- Liu, Z., Zhang, Q., Xiang, X., Yang, S., Huang, Y., and Zhu, Y. (2025). Intelligent decision and planning for unmanned surface vehicle: A review of machine learning techniques. *Ocean Eng.* 327, 120968. doi: 10.1016/j.oceaneng.2025.120968
- Ma, N., Zhang, X., Zheng, H. T., and Sun, J. (2018). "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proceedings of the European Conference on Computer Vision*, (Munich, Germany: Springer). 122–138.
- Maki, A., Fukui, K., Kawawada, Y., and Kiya, M. (2002). "Automatic ship identification in ISAR imagery: A non-line system using CMSM," in *Proceedings of the 2002 IEEE Radar Conference*, (Long Beach, CA, USA: IEEE). 206–211.
- Martin-Salinas, I., Badia, J. M., Valls, O., Leon, G., Del Amor, R., Belloch, J. A., et al. (2025). Evaluating and accelerating vision transformers on GPU-based embedded edge AI systems. *J. Supercomputing* 81, 1–21. doi: 10.1007/s11227-024-06807-1
- Molchanov, P., Tyree, S., Karras, T., Aila, T., and Kautz, J. (2016). "Pruning convolutional neural networks for resource efficient inference," in *International Conference on Learning Representations*, (Toulon, France: Curran Associates, Inc). 1–17.
- Moosbauer, S., König, D., Jäkel, J., and Teutsch, M. (2019). "A benchmark for deep learning based object detection in maritime environments," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (Long Beach, CA, USA: IEEE). 916–925.
- Ong, J., Vo, B.-T., Vo, B.-N., Kim, D. Y., and Nordholm, S. (2022). A bayesian filter for multiview 3D multi-object tracking with occlusion handling. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 2246–2263. doi: 10.1109/TPAMI.2020.3034435
- Ranftl, R., Bochkovskiy, A., and Koltun, V. (2021). "Vision transformers for dense prediction," in *IEEE International Conference on Computer Vision*, (Montreal, Canada: IEEE). 12159–12168. doi: 10.1109/ICCV48922.2021.01196
- Rensink, R. A., and Enns, J. T. (1998). Early completion of occluded objects. *Vision Res.* 38, 2489–2505. doi: 10.1016/S0042-6989(98)00051-0
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L. C. (2018). "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, (Salt Lake City, UT, USA: IEEE). 4510–4520.
- Sankaran, R. K., Paliwal, S., and Pakrashi, V. (2023). "Edge-cascade: A lightweight CNN architecture for real-time obstacle detection on USVs," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, (London, UK: IEEE). 11877–11883.
- Song, W.-J., Im, K.-S., and Lee, S.-K. (2024). A review on the USV technologies for safety surveillance and disaster prevention in the ocean. *J. Mar. Sci. Eng.* 12, 7, 1140. doi: 10.3390/jmse12071140
- Sun, F., Li, C., Xie, Y., Li, Z., Yang, C., and Qi, J. (2022). Review of deep learning applied to occluded object detection. *J. Front. Comput. Sci. Technol.* 16, 8, 1673–9418. doi: 10.3778/j.issn.1673-9418.2104046
- Tan, M., and Le, Q. V. (2019). "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the International Conference on Machine Learning*, (Long Beach, CA, USA: PMLR). 6105–6114.
- Tan, M., and Le, Q. V. (2021). "EfficientNetV2: Smaller models and faster training," in *Proceedings of the International Conference on Machine Learning*, (Vienna, Austria: PMLR). 10096–10106.
- Tang, L., Xu, Y., Xu, Z., and Zhang, W. (2022). Robust ship tracking in maritime surveillance videos under complex occlusion conditions. *IEEE Trans. Intelligent Transportation Syst.* 23, 10, 19356–19368. doi: 10.1109/TITS.2022.3142795
- Tersek, M., Zust, L., and Kristan, M. (2023). eWaSR-An embedded-compute-ready maritime obstacle detection network. *Sensors* 23, 12. doi: 10.3390/s23125386
- Wang, C.-Y., Bochkovskiy, A., and Liao, H.-Y. M. (2021). "Scaled-YOLOv4: Scaling cross stage partial network," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (Nashville, TN, USA: IEEE). 13024–13033.
- Wang, N., Wang, Y., Feng, Y., and Wei, Y. (2024a). AodeMar: Attention-aware occlusion detection of vessels for maritime autonomous surfaceships. *IEEE Trans. Intelligent Transportation Syst.* 25, 13584–13597. doi: 10.1109/TITS.2024.3398733
- Wang, N., Wang, Y., Wei, Y., Han, B., and Feng, Y. (2024b). Marine vessel detection dataset and benchmark for unmanned surface vehicles. *Appl. Ocean Res.* 142, 103835. doi: 10.1016/j.apor.2023.103835
- Wang, T., Yuan, L., Zhang, X., and Feng, J. (2019). "Distilling object detectors with fine grained feature imitation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (Long Beach, CA, USA: IEEE). 4928–4937.
- Wu, P., Huang, H., Qian, H., Su, S., Sun, B., and Zuo, Z. (2022). SRCANet: Stacked residual coordinate attention network for infrared ship detection. *IEEE Trans. Geosci. Remote Sens.* 60, 1–14. doi: 10.1109/TGRS.2022.3218563

- Wu, C. M., Lei, J., Li, Z. Q., and Ren, M. L. (2025). Ship_YOLO: General ship detection based on mixed distillation and dynamic task-aligned detection head. *Ocean Eng.* 323, 120616. doi: 10.1016/j.oceaneng.2025.120616
- Xiang, W., Pan, C., and Liu and Liu, J. Y. (2024). A ghost and attention mechanism based deep learning approach for SAR small target image detection. *Chiang Mai J. Sci.* 51, 5, e2024076. doi: 10.12982/CMJS.2024.076
- Yang, Z., Li, Y., Wang, B., Ding, S., and Jiang, P. (2022). A lightweight sea surface object detection network for unmanned surface vehicles. *J. Mar. Sci. Eng.* 10, 965. doi: 10.3390/jmse10070965
- Yun, S., Han, D., Chun, S., Oh, S. J., Yoo, Y., and Choe, J. (2019). "CutMix: Regularization strategy to train strong classifiers with localizable features," in *IEEE International Conference on Computer Vision*, (Seoul, South Korea: IEEE). 6022–6031.
- Zeng, G., Yu, W., Wang, R., and Lin, A. (2022). Research on mosaic image data enhancement and detection method for overlapping ship targets. *Control Theory Appl.* 39, 6, 1139–1148. doi: 10.7641/CTA.2021.10329
- Zhang, X., Zhou, X., Lin, M., and Sun, J. (2018). "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (Salt Lake City, UT, USA: IEEE). 6848–6856.
- Zhao, S., Zhang, S., and Zhang, L. (2018). Towards occlusion handling: object tracking with background estimation. *IEEE Trans. Cybernetics* 48, 2086–2100. doi: 10.1109/TCYB.2017.2727138